

CAHIER DE CONCEPTION

Application Web DR-DOCSCOL

Gestion des retraits de documents scolaires — OBC / DECC

Projet : Application de demande document académique

Application : DR-DOCSCOL (Document Retrieval — Documents Scolaires)

Contexte : Cameroun — OBC et DECC

Version : 1.0 — Juin 2026

Document associé : Cahier d'analyse (docs/cahier_analyse.md)

TABLE DES MATIÈRES

1. Introduction et objet du cahier de conception
 2. Architecture générale de la solution
 3. Conception fonctionnelle
 4. Conception technique
 5. Conception du modèle de données
 6. Conception de l'interface utilisateur
 7. Conception de la sécurité
 8. Conception des imports et flux batch
 9. Plan de tests et validation
 10. Conception du déploiement
 11. Conclusion et perspectives
 12. Glossaire et références techniques
-

1. INTRODUCTION ET OBJET DU CAHIER DE CONCEPTION

1.1 — Objet

Le présent **cahier de conception** traduit les résultats du **cahier d'analyse** en choix de conception concrets pour l'application **DR-DOCSCOL**. Il décrit :

- l'architecture globale retenue ;
- la conception fonctionnelle (cas d'utilisation, flux, routage) ;
- la conception technique (stack, services, API) ;
- le modèle de données ;
- les interfaces, la sécurité, les tests et le déploiement.

1.2 — Prérequis

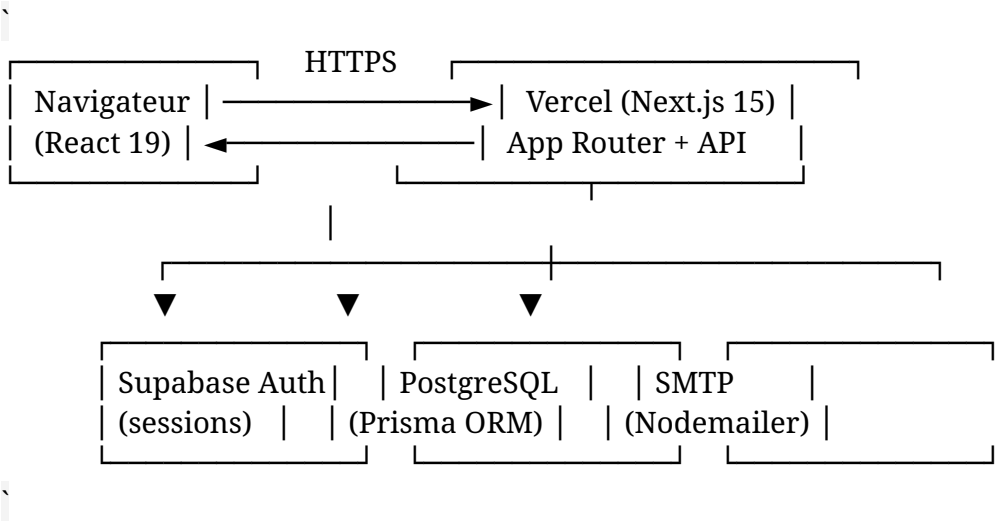
La lecture du cahier d'analyse est recommandée pour comprendre le contexte, les besoins et les contraintes ayant guidé les choix présentés ici.

1.3 — État d'implémentation

La conception décrite correspond à l'**implémentation réelle** du dépôt source (Next.js 15,

2. ARCHITECTURE GÉNÉRALE DE LA SOLUTION

2.1 — Vue d'ensemble



2.2 — Pattern architectural

Organisation MVC adaptée à Next.js App Router :

Couche	Emplacement	Rôle
Vue	app/**/page.tsx, components/	Rendu UI
Contrôleur	app/api/**, app/**/actions.ts	Entrées HTTP et Server Actions
Modèle	prisma/schema.prisma	Persistance relationnelle
Services métier	lib/*.ts	Règles métier centralisées

2.3 — Découpage modulaire

Module	Dossiers principaux
Authentification	app/auth/, lib/auth.ts, lib/supabase/
Espace élève	app/dashboard/, components/dashboard/
Back-office	app/admin/, components/admin/
Agent centre	app/centre-examen/, lib/centre-examen-service.ts
Consultation publique	app/consultation/, lib/public-consultation.ts
Documents et routage	lib/document-routing.ts, lib/document-status-transition.ts
Rendez-vous	lib/appointment-service.ts
Duplicatas	lib/duplicata-service.ts, lib/duplicata-storage.ts
Imports CSV	lib/csv-import-parser.ts, lib/admin-student-import.ts, lib/document-availability-import.ts

3. CONCEPTION FONCTIONNELLE

3.1 — Acteurs et rôles implémentés

Rôle Prisma	Espace	Fichier garde
ELEVE	/dashboard	lib/auth.ts → requireRole("ELEVE")
ADMINISTRATEUR	/admin	Périmètre organismeId + antenneRegionaleId
AGENT_CENTRE_EXAMEN	/centre-examen	Filtre région + documents centre

Sous-profils admin : connexion séparée OBC (/auth/login/obc) et DECC (/auth/login/decc).

3.2 — Diagramme de cas d'utilisation (synthèse)

UC	Acteur	Description	Route / action
UC-01	Élève	Activer compte	/auth/register
UC-02	Élève	Consulter documents	/dashboard/documents
UC-03	Visiteur	Consultation matricule	/consultation, POST /api/public/consultation
UC-04	Élève	Demander relevé	app/dashboard/ actions.ts
UC-05	Élève	Demander duplicata + payer	Dashboard + Paiement, Recu
UC-06	Élève	Réserver RDV	/dashboard/rendez-vous
UC-07	Admin	Import élèves (B)	/admin/students, importTestDataFromCsv
UC-08	Admin	Disponibilisation (A)	importDocumentAvailabilityAction
UC-09	Admin	Changer statut	/admin/documents, updateDocumentStatusAction
UC-10	Admin	Valider duplicata	updateDuplicataPieceReviewAction
UC-11	Agent	Confirmer retrait	confirm-withdrawal API
UC-12	Système	Notifier disponibilité	applyDocumentStatusTransition

3.3 — Flux métier conçus

Flux 1 — Parcours élève complet

Import B / saisie manuelle → PAS_DISPONIBLE

↓

Activation compte (/auth/register)

↓

Import A / action admin → DISPONIBLE + notification

↓
Consultation + RDV (si routage centre/antenne)
↓
Retrait confirmé → RETIRE
,

Flux 2 — Consultation publique

Étape	Conception
Entrée	Landing #consultation (QR) ou /consultation
Saisie	Matricule uniquement
Traitement	lookupPublicConsultationByMatricule()
Sortie	Statuts documents (hors duplicatas)
Actions	Liens Activer mon compte / Me connecter
Sécurité	Rate limiting par IP

3.4 — Matrice de routage (conception métier)

Implémentée dans lib/document-routing.ts :

Document	Organisme	Lieu	Confirmé par
BEPC original	DECC	Centre	Agent
BEPC relevé	DECC	Centre	Agent
BEPC duplicata	DECC	Antenne	Admin
Probatoire relevé	OBC	Centre	Agent
Probatoire duplicata	OBC	Antenne	Admin
Bac relevé	OBC	Centre	Agent
Bac original	OBC	Antenne	Admin
Bac duplicata	OBC	Antenne	Admin

3.5 — Inventaire des pages

Zone	Routes
Public	/, /consultation
Auth	/auth/login, /auth/register, /auth/login/obc, /auth/login/decc, /auth/login/centre-examen
Élève	/dashboard, /dashboard/documents, /dashboard/rendez-vous, /dashboard/payments, /dashboard/notifications
Admin	/admin, /admin/documents, /admin/students, /admin/payments, /admin/appointments, /admin/rdv-disponibilites, /admin/audit-logs
Agent	/centre-examen

Référence complète : docs/routes-implementees.md

4. CONCEPTION TECHNIQUE

4.1 — Stack retenue

Composant	Technologie	Version
Framework	Next.js App Router	15.x
UI	React, Tailwind, Radix UI	React 19
Langage	TypeScript	5.x
ORM	Prisma	6.x
BDD	PostgreSQL (Supabase)	—
Auth	Supabase Auth + @supabase/ssr	—
Validation	Zod	—
Email	Nodemailer	—
Hébergement	Vercel	Serverless

4.2 — Services métier (couche lib/)

Service	Fichier	Responsabilité
Routage	lib/document-routing.ts	OBC/DECC, antennes, scope admin
Statuts	lib/document-status-transition.ts	Transitions, notifications, audit
RDV	lib/appointment-service.ts	Créneaux, quota, fériés
Duplicata	lib/duplicata-service.ts	Barème, cooldown, sync statut
Import élèves	lib/admin-student-import.ts	Import B
Import dispo	lib/document-availability-import.ts	Import A
Parser CSV	lib/csv-import-parser.ts	Parsing partagé
Consultation	lib/public-consultation.ts	Lookup matricule
Auth	lib/auth.ts	Session, rôles
Notifications	lib/notification-service.ts, lib/mail-service.ts	In-app + email
Rate limit	lib/simple-rate-limit.ts	Protection API publique

4.3 — Conception API REST

38 handlers sous app/api/ :

Domaine	Routes clés
Santé	GET /api/health
Auth	/api/auth/login, register, me, logout
Élève	/api/students/me/documents, appointments, notifications
RDV	GET /api/appointments/slots
Admin	/api/admin/documents, students, dashboard/stats
Centre	/api/centre-examen/appointments, confirm-withdrawal
Public	POST /api/public/consultation
Interne	/api/internal/notifications/dispatch
Paielement	POST /api/payments/webhook

Helpers : lib/api-utils.ts — requireApiUser(), requireInternalRequest(), validation Zod.

4.4 — Server Actions

Fichier	Actions principales
app/auth/actions.ts	Connexion, inscription, sélection région admin
app/dashboard/actions.ts	Demandes relevé/duplicata, RDV
app/admin/actions.ts	Statuts, imports A/B, quota, fériés, duplicata
app/account/actions.ts	Profil utilisateur

4.5 — Middleware et session

- `middleware.ts` : rafraîchissement session Supabase, protection routes privées.
- Cookies `sb-*` vérifiés pour `/dashboard`, `/admin`, `/account`.

5. CONCEPTION DU MODÈLE DE DONNÉES

Source : `prisma/schema.prisma` — 20 migrations versionnées.

5.1 — Entités principales

Modèle	Table	Rôle
User	users	Élève, admin, agent
Organisme	organismes	OBC, DECC
AntenneRegionale	antennes_regionales	Périmètre régional
CentreExamen	centres_examen	Agent rattaché
ExamenValide	examens_valides	Examen par élève/diplôme
DocumentAcademique	documents	Document + statut
Duplicata	duplicatas	Dossier duplicata
PieceDuplicata	pieces_duplicata	Justificatifs
RendezVous	rendez_vous	RDV retrait
DisponibiliteRdv	disponibilites_rdv	Créneaux
Paielement	paiements	Transaction
Recu	recus	Reçu généré
Notification	notifications	Alertes in-app
AuditLog	audit_logs	Traçabilité
ParametreRendezVous	parametres_rdv	Quota global

5.2 — Énumérations

Enum	Valeurs
Role	ELEVE, ADMINISTRATEUR, AGENT_CENTRE_EXAMEN
StatutDocument	PAS_DISPONIBLE, DISPONIBLE, RETIRE
TypeDocument	ORIGINAL, RELEVÉ_NOTES, DUPLICATA
DiplomePrincipal	BEPC, PROBATOIRE, BACCALAUREAT
StatutRendezVous	PLANIFIE, CONFIRME, ANNULE, HONORE
StatutValidationDuplicata	SOUMISE, EN_ANALYSE, VALIDEE, REJETEE

5.3 — Conception des transitions de statut

Service : `lib/document-status-transition.ts`

Transition	Acteur	Règle
→ DISPONIBLE	Admin	Duplicata : validation VALIDEE requise
→ RETIRE (centre)	Agent	Routage CENTRE_EXAMEN uniquement
→ RETIRE (antenne)	Admin	Duplicata, Bac original
PAS_DISPONIBLE ↔ DISPONIBLE	Admin	Périmètre organisme/antenne

Effets collatéraux conçus : notification email, AuditLog, sync duplicata, RDV → HONORE.

5.4 — Conception duplicata et paiement

Élément	Conception
Tarif relevé duplicata	10 000 FCFA
Tarif original duplicata	15 000 FCFA
Cooldown	1 an après RETIRE
Pièces	4 types enum TypePieceDuplicata
Stockage	Bucket Supabase duplicata-documents (MVP partiel)
Paieement	MVP simulé, webhook /api/payments/webhook

6. CONCEPTION DE L'INTERFACE UTILISATEUR

6.1 — Charte graphique

- Palette ardoise / ambre : docs/CHARTES_GRAPHIQUES_COULEURS.md
- Logo : components/ui/DocScolLogo.tsx
- Tokens CSS : styles/globals.css, tailwind.config.js

6.2 — Composants structurants

Composant	Fichier	Usage
Shell dashboard	components/dashboard/dashboard-shell.tsx	Layout mobile-first
Header	components/dashboard/dashboard-header.tsx	Navigation
List panel	components/dashboard/dashboard-list-panel.tsx	Listes responsives
Landing	components/landing/landing-page.tsx	Page d'accueil
Consultation QR	components/landing/consultation-access.tsx	Section QR
Import admin	components/admin/admin-availability-import-form.tsx	Import A
Dialogues	components/ui/dialog.tsx, sheet.tsx	Modales scrollables

6.3 — Parcours UI par profil

Profil	Écrans conçus
Visiteur	Hero, problème/solution, QR consultation, FAQ
Élève	Dashboard, documents, RDV, paiements, notifications
Admin	Documents, élèves (imports A/B), audit, RDV OBC
Agent	Liste RDV centre, bouton confirmation retrait

6.4 — Responsive

- `h-dvh + scroll <main>` dans le shell dashboard.
- Cartes empilées sur mobile (`rendez-vous/page.tsx`).
- Dialogues avec overflow scroll pour petits écrans.

7. CONCEPTION DE LA SÉCURITÉ

7.1 — Authentification

Étape	Conception
Credentials	Supabase Auth (email + mot de passe)
Lien profil	<code>User.authUserId</code> unique
Activation	Matricule + email préexistants en Prisma
Connexion	Matricule + email + mot de passe

7.2 — Autorisation (defense in depth)

Niveau	Mécanisme
Edge	<code>middleware.ts</code>
Page	<code>requireUser()</code> , <code>requireRole()</code>
API	<code>requireApiUser(role?)</code>
Métier	<code>canAdminAccessDocument()</code> , <code>getCentreDocumentWhere()</code>
Interne	Header <code>INTERNAL_API_SECRET</code>
Public	Rate limit consultation

7.3 — Traçabilité

- `AuditLog` : changements statut, imports, validations.
- `MailLog` : succès/échec emails.
- Mots de passe **jamais** stockés en Prisma.

8. CONCEPTION DES IMPORTS ET FLUX BATCH

8.1 — Import B — Nouveaux élèves

Élément	Conception
En-tête CSV	<code>STUDENT_IMPORT_CSV_HEADER</code> (sans colonne

	statut)
Modèle	public/templates/import-eleves.csv
Action	importTestDataFromCsv → upsertStudentImportRow
Statut document	Toujours PAS_DISPONIBLE
UI	AdminStudentsImportForm

8.2 — Import A — Disponibilisation

Élément	Conception
En-tête CSV	AVAILABILITY_IMPORT_CSV_HEADER
Modèle	public/templates/import-disponibilisation.csv
Action	importDocumentAvailabilityFromCsv
Règle	1 ligne = 1 document → DISPONIBLE
Prérequis	Élève + document existants
Exclusion	Duplicatas
Effet	Notification via applyDocumentStatusTransition
UI	AdminAvailabilityImportForm
Revalidation	/admin/documents, /dashboard/documents

8.3 — Gestion des erreurs import

- Erreurs collectées **par ligne** (matricule inconnu, document absent, type invalide).
- Retour URL : availStatus, availMessage, availErrors.
- Affichage détaillé dans le formulaire admin.

9. PLAN DE TESTS ET VALIDATION

9.1 — Stratégie de test conçue

Niveau	Méthode	Outils / docs
Manuel fonctionnel	Scénarios par rôle	docs/ GUIDE_TEST_FONCTIONNEL.md
API	Requêtes curl	docs/ CURL_TEST_EXAMPLES.md
Données	Seeds et CSV	Scripts npm run seed:*
Post-implémentation	Checklist	docs/ CHECKLIST_POST_IMPLEMENTATION.md

9.2 — Scénarios critiques

ID	Scénario	Résultat attendu
T-01	Consultation matricule valide	Statuts affichés, pas de duplicatas
T-02	Import B nouvel élève	Document PAS_DISPONIBLE
T-03	Import A ligne valide	DISPONIBLE + notification

T-04	RDV document DISPONIBLE	Créneau réservé
T-05	Retrait agent centre	RETIRE + RDV HONORE
T-06	Duplicata non validé → DISPONIBLE	Erreur métier
T-07	Admin hors périmètre	Accès refusé

9.3 — Jeux de test

- Comptes : docs/connexions-tests-completes.md
- 5 élèves demo : npm run seed:soutenance-eleves + docs/demo/KIT_DEMO_COMPLET.pdf
- Modèles CSV : public/templates/

10. CONCEPTION DU DÉPLOIEMENT

10.1 — Architecture de déploiement

Composant	Cible
Application	Vercel (serverless)
Base	Supabase PostgreSQL
Auth	Supabase Auth
Email	SMTP externe

10.2 — Configuration Vercel

Fichier vercel.json :

- installCommand : npm ci
- buildCommand : npm run vercel-build

Script build :

```
`
prisma generate && next build && node scripts/copy-prisma-engines.mjs
`
```

10.3 — Variables d'environnement

Variable	Usage
DATABASE_URL	Prisma pooler
DIRECT_URL	Migrations
NEXT_PUBLIC_SUPABASE_URL	Client Supabase
NEXT_PUBLIC_SUPABASE_ANON_KEY	Clé publique
SUPABASE_SERVICE_ROLE_KEY	Admin Supabase
INTERNAL_API_SECRET	Routes internes / webhook
NEXT_PUBLIC_SITE_URL	URL QR consultation

10.4 — Commandes de déploiement

```
`bash
vercel --prod      # Production
npm run db:migrate:deploy # Migrations BDD
```

npm run vercel-build # Build local identique CI

10.5 — Limites de conception connues

Point	Statut
Pooler Prisma/Supabase	Stabilisation production à finaliser
Paielement réel	Conception webhook prête, prestataire à brancher
Storage duplicata	Bucket défini, intégration complète TBD
Tests E2E automatisés	Non conçus dans le MVP

11. CONCLUSION ET PERSPECTIVES

11.1 — Bilan de la conception

La conception de DR-DOCSCOL repose sur :

- un **routage documentaire centralisé** ;
- **deux pipelines CSV distincts** ;
- une **consultation publique minimale** ;
- une **architecture Next.js + Prisma + Supabase** déployable sur Vercel ;
- une **traçabilité** systématique des actions sensibles.

11.2 — Évolutions prévues

Évolution	Impact conception
Paielement Mobile Money	Brancher prestataire sur webhook existant
Storage pièces	Finaliser Supabase Storage
Tests E2E	Suite Playwright
Statistiques avancées	Nouveaux endpoints admin
Traduction diplôme	Nouveau module documentaire

12. GLOSSAIRE ET RÉFÉRENCES TECHNIQUES

12.1 — Glossaire technique

Terme	Définition
App Router	Système de routage Next.js 13+ basé sur app/
Server Action	Fonction serveur "use server" appelée depuis le client
Prisma	ORM TypeScript pour PostgreSQL
Scope admin	Filtre organismeId + antenneRegionaleId
Revalidate	Invalidation cache Next.js après mutation

12.2 — Fichiers de référence

Ressource	Chemin
-----------	--------

Schéma BDD	prisma/schema.prisma
Routage métier	lib/document-routing.ts
Transitions statut	lib/document-status-transition.ts
Routes	docs/routes-implementees.md
API	docs/guide_api_mis_a_jour.md
UML	docs/diagrammes-images/
Cahier d'analyse	docs/cahier_analyse.md

Fin du cahier de conception — DR-DOCSCOL v1.0